

```

METHOD /iwbsp/if_mgw_appl_srv_runtime~get_expanded_entityset.
**TRY.
*CALL METHOD SUPER->/IWBEF/IF_MGW_APPL_SRV_RUNTIME~GET_EXPANDED_ENTITYSET
**
** EXPORTING
**     iv_entity_name           =
**     iv_entity_set_name      =
**     iv_source_name          =
**     it_filter_select_options =
**     it_order                 =
**     is_paging                =
**     it_navigation_path      =
**     it_key_tab               =
**     iv_filter_string         =
**     iv_search_string         =
**     io_expand                =
**     io_tech_request_context =
**
** IMPORTING
**     er_entityset            =
**     et_expanded_clauses     =
**     et_expanded_tech_clauses =
**     es_response_context     =
**
**
** CATCH /iwbsp/cx_mgw_busi_exception .
** CATCH /iwbsp/cx_mgw_tech_exception .
**ENDTRY.

```

```

DATA: lt_so_all TYPE STANDARD TABLE OF zcl_zso_creation_mpc_ext=>ty_so_all.

```

```

CASE iv_entity_set_name.
  WHEN 'soheaderSet'.

```

```

    " 1. Fetch Header Data (Max 10 rows)
    SELECT FROM vbak
      FIELDS vbeln, erdat, erzet, ernam, auart, netwr,
             waerk, vkorg, vtweg, spart, kunnr
    INTO TABLE @DATA(lt_header)
    UP TO 10 ROWS.

```

```

    IF sy-subrc EQ 0.
      " 2. Fetch Customer Data
      SELECT FROM kna1
        FIELDS kunnr, name1, land1
        FOR ALL ENTRIES IN @lt_header
        WHERE kunnr = @lt_header-kunnr
        INTO TABLE @DATA(lt_customer).
    
```

```

    " 3. Fetch Item Data
    SELECT FROM vbap
      FIELDS vbeln, posnr, matnr, arktx, zmeng, meins,
             netwr, waerk, werks, lgort, vstel, route
    FOR ALL ENTRIES IN @lt_header
    WHERE vbeln = @lt_header-vbeln
    INTO TABLE @DATA(lt_item).

```

```

    IF sy-subrc EQ 0.
      " 4. Fetch Material/Product Data
      SELECT FROM mara
        FIELDS matnr, mtart
        FOR ALL ENTRIES IN @lt_item
        WHERE matnr = @lt_item-matnr
        INTO TABLE @DATA(lt_material).
    
```

```
ENDIF.  
ENDIF.
```

```
" 5. Deep Entity Set Construction
```

```
lt_so_all = VALUE #(  
  FOR ls_header IN lt_header  
  (  
    vbeln = ls_header-vbeln  
    erdat = ls_header-erdat  
    erzet = ls_header-erzet  
    ernam = ls_header-ernam  
    auart = ls_header-auart  
    netwr = ls_header-netwr  
    waerk = ls_header-waerk  
    vkorg = ls_header-vkorg  
    vtweg = ls_header-vtweg  
    spart = ls_header-spart  
    kunnr = ls_header-kunnr  
  
    " Explicit mapping for customer to avoid type conflicts  
    tocustomer = VALUE #(  
      kunnr = ls_header-kunnr  
      name1 = VALUE #( lt_customer[ kunnr = ls_header-kunnr ]-name1 OPTIONAL )  
      land1 = VALUE #( lt_customer[ kunnr = ls_header-kunnr ]-land1 OPTIONAL )  
    )  
  
    " Build inline items array  
    toitem = VALUE #( FOR ls_item IN lt_item WHERE ( vbeln = ls_header-vbeln )  
      (  
        vbeln = ls_item-vbeln  
        posnr = ls_item-posnr  
        matnr = ls_item-matnr  
        arktx = ls_item-arktx  
        zmeng = ls_item-zmeng  
        meins = ls_item-meins  
        netwr = ls_item-netwr  
        waerk = ls_item-waerk  
        werks = ls_item-werks  
        lgort = ls_item-lgort  
        vstel = ls_item-vstel  
        route = ls_item-route  
  
        " Nested product mapping  
        toproduct = VALUE #(  
          matnr = ls_item-matnr  
          mtart = VALUE #( lt_material[ matnr = ls_item-matnr ]-mtart OPTIONAL )  
        )  
      )  
    )  
  )  
).  
  
*  
*  
*  
*
```

```
" 6. Export reference back to OData Framework
```

```
copy_data_to_ref(  
  EXPORTING  
    is_data = lt_so_all  
  CHANGING  
    cr_data = er_entityset ).
```

```
ENDCASE.  
ENDMETHOD.
```

